

Software Design Document

High-Level & Low-Level Design Specifications for PricePilot AI Enterprise Platform

Document ID:	DOC-SDD-PRICEPILOT-2026-V2
Release Version:	Version 2.0.0
Platform:	PricePilot AI — AI-Powered Dynamic Pricing & Demand Forecasting SaaS
Technology Stack:	Python 3.13 · FastAPI · React 19 · Scikit-Learn · PostgreSQL · SQLAlchemy
ML Algorithm:	Extra Trees Regressor (Extremely Randomized Trees) — R^2 0.9650
Authors:	Narendar Reddy (Lead) · Manvitha · Pravallika · Ashwindh
Organization:	Infosys Springboard 7.0 Internship Program
Completion:	August 2026

CONFIDENTIALITY NOTICE: This document is proprietary intellectual property of the PricePilot AI Team submitted for Infosys Springboard 7.0. Unauthorized reproduction or distribution is prohibited.

DOCUMENT REVISION HISTORY

Rev	Date	Author	Reviewer	Description
1.0	2026-08-01	Team PricePilot	Infosys Mentor	Initial Draft — Project Inception
1.5	2026-08-04	Narendar Reddy	Manvitha	Mid-Sprint Review — ML Pipeline Integration
2.0	2026-08-07	Full Team	Academic Committee	Final Production Release — University Submission

1.0 INTRODUCTION & SCOPE

This Software Design Document (SDD) provides a comprehensive architectural, component-level, and data-flow specification for PricePilot AI. The system is a four-tier SaaS enterprise platform combining a React 19 Single Page Application (SPA) frontend, Python 3.13 FastAPI backend microservices, Neon Cloud PostgreSQL relational database, and a Scikit-Learn Extra Trees Regressor ML engine serialized via joblib.

2.0 HIGH-LEVEL DESIGN (HLD) — SYSTEM OVERVIEW

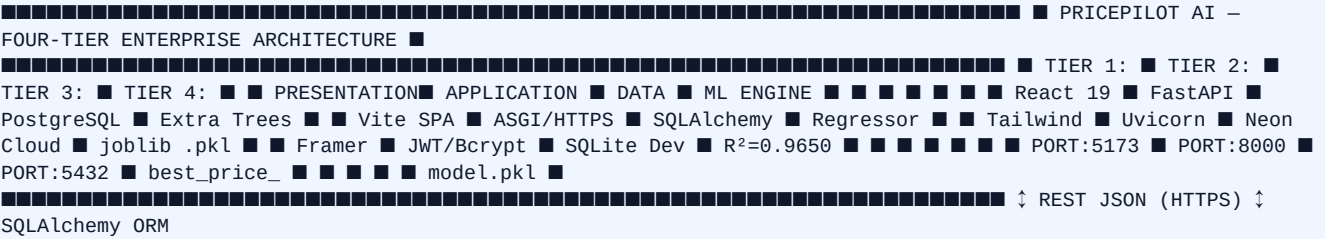


Figure 2.1 — PricePilot AI Four-Tier Architecture Diagram

3.0 COMPONENT ARCHITECTURE DESIGN

3.1 Frontend Component Hierarchy (React 19 SPA)

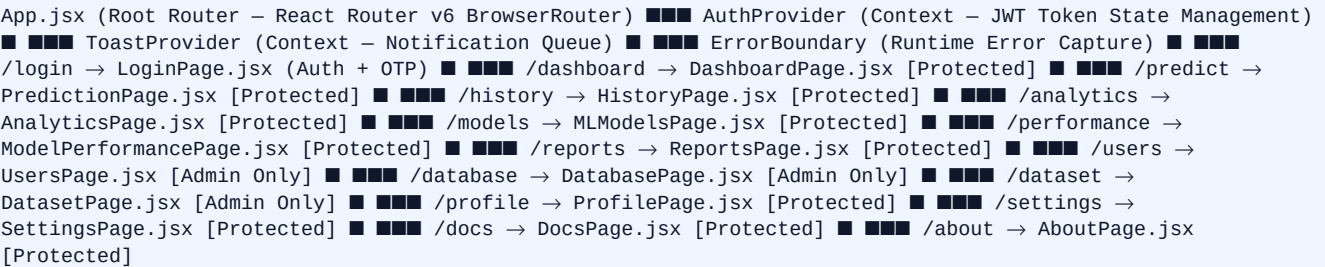


Figure 3.1 — Frontend Component Tree Diagram

3.2 Backend Router & Module Architecture (FastAPI)

```

main.py (FastAPI ASGI Application Entry Point) ■■■ SecurityHeadersMiddleware → middleware.py ■■■
CORSMiddleware → Origin Allowlist ■■■ StaticFiles → /static → backend/static/documents/ ■■■ auth.router →
routers/auth.py ■ ■■■ POST /api/auth/register (UserRegister schema) ■ ■■■ POST /api/auth/login (UserLogin
schema → JWT Token) ■ ■■■ POST /api/auth/forgot-password (OTPRequest schema) ■ ■■■ POST
/api/auth/verify-otp (OTPVerify schema) ■ ■■■ POST /api/auth/reset-password (PasswordReset schema) ■ ■■■
GET /api/auth/me (Bearer JWT → UserResponse) ■■■ users.router → routers/users.py ■ ■■■ GET /api/users
(Admin – User Registry List) ■ ■■■ POST /api/users (Admin – Create User) ■ ■■■ PUT /api/users/{id} (Admin –
Update User) ■ ■■■ DELETE /api/users/{id} (Admin – Delete User) ■■■ admin_router → routers/users.py ■ ■■■
POST /api/users/bulk-status (Admin – BulkStatusRequest) ■ ■■■ POST /api/users/bulk-delete (Admin –
BulkDeleteRequest) ■ ■■■ GET /api/admin/export-users (Admin – openpyxl .xlsx Binary) ■■■ predict.router →
routers/predict.py ■ ■■■ POST /api/predict (ProductFeatures → ML Inference) ■ ■■■ GET /api/model-status
(Model Health Check) ■■■ dashboard.router → routers/dashboard.py ■ ■■■ GET /api/dashboard/stats ■ ■■■ GET
/api/dashboard/recent-activity ■■■ docs.router → routers/docs.py ■ ■■■ GET /api/docs ■ ■■■ GET
/api/docs/{doc_id} ■ ■■■ GET /api/docs/download/{doc_id}

```

Figure 3.2 — Backend Router Architecture Diagram

4.0 AUTHENTICATION SYSTEM DESIGN

JWT Authentication Flow: User FastAPI /auth Database ■ ■ ■ ■ ■ POST /login ■ ■ ■ ■ ■ {username, password} ■ ■ ■ SELECT user ■ ■ ■ ■ ■ user record ■ ■ ■ ■ ■ Bcrypt.verify() ■ ■ ■ (12 salt rounds) ■ ■ ■ ■ ■ jwt.encode() ■ ■ ■ (HS256, 30min TTL) ■ ■ ■ ■ ■ 200 {access_token} ■ ■ ■ ■ ■ GET /api/predict ■ ■ ■ ■ ■ Authorization: Bearer ■ ■ ■ ■ ■ jwt.decode() ■ ■ ■ ■ ■ Verify signature ■ ■ ■ ■ ■ 200 {prediction} ■ ■ ■ ■ ■

Figure 4.1 — JWT Authentication Sequence Flow

4.1 OTP Password Reset Design

OTP Recovery Flow: User /auth/forgot-password DB: password_reset_otps ■ ■ ■ ■ ■ POST identifier ■ ■ ■ ■ ■ ■ ■ ■ ■ SELECT user ■ ■ ■ ■ ■ ■ ■ ■ ■ Generate 6-digit OTP ■ ■ ■ SET expires_at = +15min ■ ■ ■ ■ ■ INSERT otp_record ■ ■ ■ ■ ■ ■ ■ ■ ■ 200 OTP Issued ■ ■ ■ ■ ■ ■ ■ ■ ■ POST /verify-otp ■ ■ ■ ■ ■ {identifier, ■ ■ ■ ■ ■ SELECT WHERE ■ ■ otp_code, ■ otp_code = ? AND ■ ■ new_password} ■ expires_at > NOW() ■ ■ ■ ■ ■ valid record ■ ■ ■ ■ ■ ■ ■ ■ ■ Bcrypt new_password ■ ■ ■ ■ ■ UPDATE user.password ■ ■ ■ ■ ■ UPDATE otp.is_used=True ■ ■ ■ ■ ■ 200 Reset OK ■ ■ ■ ■ ■

Figure 4.2 — OTP Password Reset Sequence Flow

5.0 MACHINE LEARNING PREDICTION DESIGN

```
ML Prediction Pipeline Design: Client FastAPI Database ■ ■ ■ ■ ■ POST /api/predict ■ ■ ■ ■ ■
ProductFeatures (16 ■ Pydantic validation ■ ■ attributes) ■ input_df = DataFrame() ■ ■ ■ ■ ■
model.predict(input_df) ■ ■ ■ ← Extra Trees .pkl ■ ■ ■ (best_price_model.pkl) ■ ■ ■ ■ ■ predicted_price
■ ■ ■ profit_margin = 35% ■ ■ ■ estimated_cost = 65% ■ ■ ■ confidence = 0.965 ■ ■ ■ ■ ■ INSERT
predictions ■ ■ ■ ■ ■ INSERT history ■ ■ ■ ■ ■ INSERT activity_log ■ ■ ■ ■ ■ 200
{predicted_price, ■ ■ ■ profit_margin, ■ ■ ■ confidence_score} ■ ■
```

Figure 5.1 — ML Prediction Sequence Flow Diagram

6.0 DATABASE DESIGN OVERVIEW

Table Name	Primary Key	Foreign Keys	Key Columns	Purpose
users	id (PK)	—	email (UQ), username (UQ), role, status	User account registry
predictions	id (PK)	products.id, users.id	predicted_price, confidence_score, model_name	ML prediction records
prediction_history	id (PK)	predictions.id, users.id	input_data (JSON), predicted_price	Full audit log of predictions
products	id (PK)	—	name, category, current_price, cost_price	Product catalog
price_recommendations	id (PK)	products.id	recommended_price, forecasted_demand	AI price advisory
demand_forecasts	id (PK)	products.id	predicted_demand, lower_bound, upper_bound	Demand volume forecast
activity_logs	id (PK)	users.id	action (VARCHAR 255), timestamp	System audit trail
password_reset_otps	id (PK)	users.id	otp_code (6 digits), expires_at, is_used	OTP reset tokens
notifications	id (PK)	—	title, message, type, is_read	System notifications
settings	id (PK)	—	theme, language, notifications_enabled	System configuration

Table 6.1 — PostgreSQL Database Schema Summary